Data Manipulation (continued)

We illustrate the SQL statements using the instance of the following table:
Staff (staffNo, fName, lName, city, position, salary, branchNo)

Staff

staffNO	fName	lName	city	position	salary	branchNo
s1	Gad	Ahmed	Tanta	Manager	60,000	B02
s2	Samy	Tamer	Cairo	Assistant	30,000	B02
s3	Samy	Ahmed	Tanta	Supervisor	50,000	B01
s4	Tamer	Samir	Giza	Manager	40,000	B03

Row selection (WHERE clause)

- The previous section shows the use of the SELECT statement to retrieve all rows from a table.
- However, we often need to restrict the rows that are retrieved.
  This can be achieved with the **WHERE** clause, which consists of the keyword **WHERE** followed by a search condition that specifies the rows to be retrieved.

A. ***Pattern match:*** Test whether a string matches a specified pattern.
B. ***Null:*** Test whether a column has a null (unknown) value.

We now present examples of each of these types of search conditions.

A. Pattern Match search condition (LIKE/NOT LIKE)

SQL has two special pattern-matching symbols:

- **%** :Percent character represents any sequence of **zero or more** characters (*wildcard*).
- **_** :Underscore character represents **any single** character.

All other characters in the pattern represent themselves.
For example:

1. address LIKE 'H%'
   Means the first character must be *H*, but the rest of the string can be anything.
2. address LIKE 'H_ _ _'
   Means that there must be exactly four characters in the string, the first of which must be an *H*.
3. address LIKE '%e'
   Means any sequence of characters, of length at least 1, with the last character an *e*.
4. address LIKE '%Glasgow%'
   Means a sequence of characters of any length containing *Glasgow*.
5. address NOT LIKE 'H%'
   Means the first character cannot be an *H*.

Ex 1: Pattern match Search Condition (LIKE/NOT LIKE)

Find all the staff whose names begin with the letter "G" or "T".

Solution:
Using the pattern-matching search condition of SQL, we can find all staffs with the first name that starts with either "G" or "T" using the following query.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE fName LIKE'G%' OR fName LIKE 'T%';
```

Table 1 Result table for Example 1.

staffNo	fName	lName	position
s1	Gad	Ahmed	Manager
s4	Tamer	Samir	Manager

B. NULL search Condition (IS NULL/IS NOT NULL)

Ex 2: NULL search condition (IS NULL/IS NOT NULL)
List the details of all staff where a branch number has not been supplied.

Staff

staffNO	fName	lName	city	position	salary	branchNo
s1	Gad	Ahmed	Tanta	Manager	60,000	B02
s2	Samy	Tamer	Cairo	Assistant	30,000	
s3	Samy	Ahmed	Tanta	Supervisor	50,000	B01
s4	Tamer	Samir	Giza	Manager	40,000	B03

Solution:
From the Staff table, we can see that there are two staff members without *branch number*.

- A null is considered to have an unknown value, so we must test for null explicitly using the special keyword **IS NULL**:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo IS NULL;
```

The result table is shown in Table 2 The negated version (IS NOT NULL) can be used to test for values that are not null.

Table 2 Result table for Example 2.

staffNo	fName	lName	position
s2	Samy	Tamer	Assistant

Sorting Results (ORDER BY Clause)

- We can ensure the results of a query are sorted using the ORDER BY clause in the SELECT statement.
  The ORDER BY clause consists of a list of **column identifiers** that the result is to be sorted on, separated by commas.
- The ORDER BY clause allows the retrieved rows to be ordered in ascending (ASC) or descending (DESC) order on any column or combination of columns, regardless of whether that column appears in the result.
- The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

Ex 1: Single-column ordering

Produce a list of salaries for all staff, arranged in descending order of salary.

Solution:

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY salary DESC;
```

The result table is shown in Table 1.

Table 1 Result table for Example 1.
Staff

staffNO	fName	lName	salary
s1	Gad	Ahmed	60,000
s3	Samy	Ahmed	50,000
s4	Tamer	Samir	40,000
s2	Samy	Tamer	30,000

Ex 2: Multiple column ordering

Produce a list of all staff sorted descending by the " lName "

Solution:

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY lName DESC;
```

In this case, we get the result table shown in Table 2.

Table 2 Result table for Example 2 with one sort key.

staffNO	fName	lName	salary
s2	Samy	Tamer	30,000
s4	Tamer	Samir	40,000
s1	Gad	Ahmed	60,000
s3	Samy	Ahmed	50,000

To produce a list of all staff sorted descending by the " lName " and ascending by the " salary " column, we specify a minor order, as follows:

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY lName DESC, salary;
```

- Now, the result is ordered first by lName, in descending alphabetic order
- And within last name, in ascending order of salary (ASC being the default setting).

Table 3 Result table with two sort keys.

staffNO	fName	lName	salary
s2	Samy	Tamer	30,000
s4	Tamer	Samir	40,000
s3	Samy	Ahmed	50,000
s1	Gad	Ahmed	60,000